

COPPER: Committed Pointer-Provenance Prefetching

Anonymous Artifact Submission

Abstract

Data-derived prefetchers can reduce pointer-chasing latency, but they can also turn address-shaped data into prefetch authority before the program has proved that the data is a pointer. COPPER addresses that authority problem with committed pointer-provenance prefetching: it records pointer-source evidence only after architectural commit, invalidates stale or mismatched evidence, and gates later data-derived prefetch issue on that committed evidence. This submission frames COPPER as an evidence-bounded reproducible hardware mechanism study evaluated with CI-proven RTL simulation, raw gem5 full-system provenance where retained, validated imported gem5 ARM-system summaries where PASS, independent simulator evidence, accepted-core-wrapper/PicoRV32 tiny-SoC full-core FPGA mapped PPA, and tool-estimated/proxy power. The supported claim is auditable at the stated evidence levels. It does not claim state-of-the-art performance, production readiness, a complete clone-local gem5 campaign, production ARM/OoO integration, production-core mapped timing, measured silicon power efficiency, ASIC signoff, foundry signoff, or silicon signoff.

1 Introduction

Pointer-heavy programs expose memory latency that ordinary address-stream prefetchers can miss. Data-derived prefetching can help by following loaded values, but the same behavior is risky when a value merely resembles an address. COPPER changes the authority rule: a prefetch candidate needs committed pointer-source evidence before issue. The mechanism is meant to filter unsafe or stale candidates while preserving the useful subset of pointer-derived prefetches.

This paper makes an evidence-bounded contribution. COPPER is evaluated as a reproducible hardware mechanism, not as a silicon product or a production processor feature. The evidence consists of executable model checks, deterministic cycle-model and core-integrated rows, source-backed independent simulator rows, validated gem5 ARM-system summaries and retained raw gem5 provenance where available, CI-proven RTL unit simulation, accepted-core-wrapper/PicoRV32 tiny-SoC full-core mapped FPGA PPA, FPGA tool-estimated power, and proxy energy. The artifact keeps these evidence levels separate and keeps negative rows visible.

This paper does not claim silicon-proven status, ASIC/foundry signoff, measured silicon power, production ARM/OoO/TLB/cache/coherence integration, state-of-the-art silicon efficiency, universal speedup, production readiness, or full silicon PPA. The claim ledger and audits are part of the submission: if the paper drifts into unsupported language, the artifact should fail.

2 Motivation

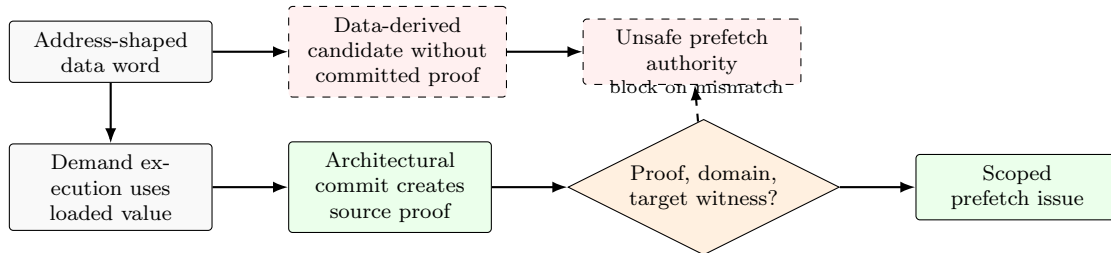


Figure 1: Pointer-chasing authority problem and COPPER’s evidence-gated alternative. A data value alone is not enough authority; issue is allowed only after committed source proof and target checks agree.

The motivating failure mode is not ordinary speculation alone. It is that a prefetcher may treat address-shaped data as authority, a concern sharpened by recent data-memory-dependent prefetcher attacks [1, 3]. COPPER instead

treats committed pointer use as authority.

3 Background And Related Work

Prior work covers next-line, stream, stride, indirect, pointer-chase, runahead, helper-thread, and dependence-guided prefetching [8, 6, 4, 10, 13], as well as metadata and capability mechanisms [2]. COPPER’s distinction is narrower: committed source-word proof gates data-derived prefetch issue. The repository matrix records what COPPER does not claim.

Recent DMP attack and vendor material make the authority problem concrete: a data-dependent prefetcher may examine memory data and create cache-visible effects from the derived target [1, 3, 7]. Software and OS defenses such as memory representation transformation or scheduler-managed prefetcher disablement address adjacent deployment points [12, 11]. Shadow-structure speculation defenses isolate speculative side effects until commit; COPPER instead scopes the authority of a non-speculative data-derived prefetch path [9].

Table 1: Related Work Positioning

Adjacent area	Typical focus	COPPER distinction	Claim boundary
Data-memory-dependent attacks	Show that data values can trigger observable prefetch behavior	Replaces address-shapedness with committed source-word authority	Attack surface is the DMP authority path, not all channels
Pointer-chase and indirect prefetching	Expose irregular memory parallelism from pointer-like streams	Allows issue only after the source word has committed-use proof	Does not claim pointer prefetching itself
Confidence and usefulness filters	Suppress candidates predicted to be inaccurate or wasteful	Blocks candidates that lack architectural authority even when they look useful	Performance regressions remain visible per row
Capability, tag, and taint mechanisms	Track architectural permissions, memory safety, or information flow	Tracks a small positive proof for microarchitectural DMP dereference permission	Does not replace architectural memory-safety mechanisms
Software or OS DMP defenses	Transform data or disable prefetching in sensitive regions	Keeps the hardware gate active at source-word granularity	Does not make compiler or OS policies obsolete
Runahead and translation aids	Expose misses or accelerate address translation	Recursive and cross-page issue stay bound to source proof and target witnesses	Not a general runahead or translation accelerator

4 Threat Model And Security Contract

COPPER targets data-memory-dependent prefetch issue where address-shaped data, stale pointer values, or mismatched address-space state could otherwise create cache-visible prefetch activity before the program has committed a pointer use. The adversary model is scoped to the prefetch authority decision: an adversary may influence data values and observe cache or memory-system side effects, while ordinary demand accesses, branch predictor channels, non-DMP microarchitectural channels, and full operating-system isolation policy are outside this mechanism-level contract.

The security contract is intentionally small: a data-derived candidate may issue only when committed source proof exists, the source proof remains clean and current, the target authority check succeeds, and the candidate is still bound to the same relevant context. Table 2 maps the main reviewer attack surfaces to the local evidence that exercises the rule.

Table 2: Security Contract And Evidence

Reviewer concern	COPPER contract response	Evidence anchor
Address-shaped data at rest	No committed source proof, so the issue gate blocks data-only candidates	COPPER_SECURITY_COVERAGE_MATRIX.md; model_tests.csv; rtl_simulation.csv
Stale source after write or coherence activity	Source proof is cleared or fails epoch/value freshness checks before issue	COPPER_AUTHORITY_REGRESSION_SUMMARY.md; COPPER_CEPF_LINE_E2E_SVA_SUMMARY.md
Recursive or cross-page target issue	Target-line witness and translation/permission checks are required before issue	COPPER_CTLW_FULL_AUTHORITY_E2E_SUMMARY.md; COPPER_CAVI_AUTHORITY_ISSUE_GATE_RTL_SUMMARY.md
Context remap, permission, or revocation	Domain and revocation checks are part of the authority path, and missing evidence blocks the candidate	COPPER_TLB_COHERENCE_AUTHORITY_FILTER_RTL_SUMMARY.md; COPPER_SARI_CLPD_CTLW_AUTHORITY_E2E_SUMMARY.md
Broader microarchitectural isolation	Out of scope for this artifact; stronger production and silicon claims remain blocked by the claim ledger	COPPER_CLAIM_LEDGER.md; stronger_claim_audit.csv

5 COPPER Design

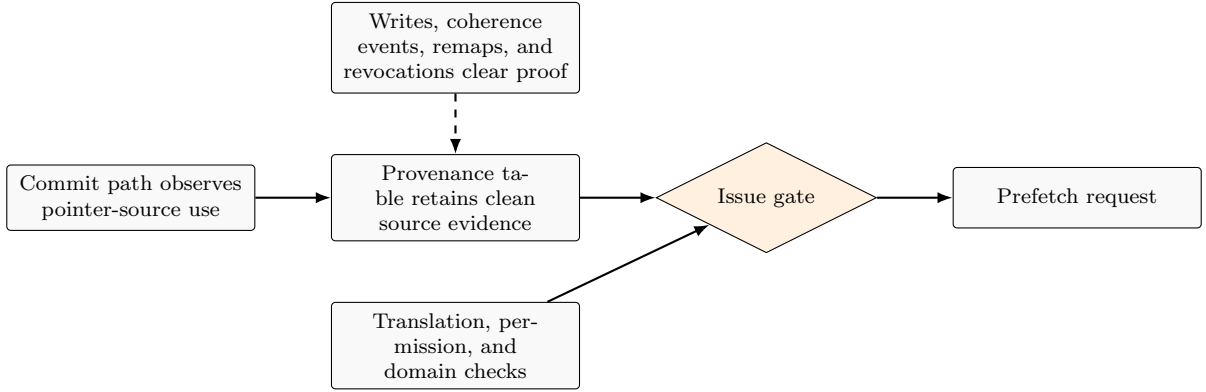


Figure 2: COPPER architecture. The commit path creates source proof, invalidation removes stale proof, and the prefetch path issues only when source proof and target authority both hold.

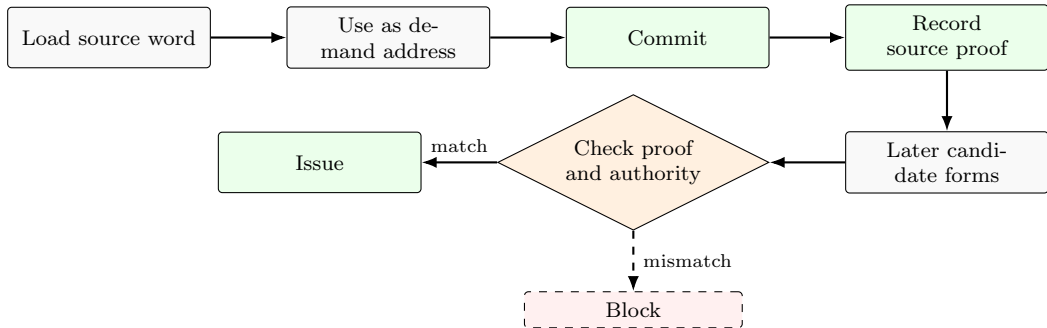


Figure 3: Committed provenance update timeline.

COPPER has three core rules: proof creation occurs after committed architectural evidence; writes, coherence updates, failed permissions, and context mismatch block or destroy proof; recursive issue does not gain authority merely because a line arrived by prefetch.

6 Implementation

The artifact contains a Python model, a trace-driven evaluation harness, SystemVerilog RTL units, C/C++ workload sources, reproduction scripts, and summary parsers. The open-source RTL smoke target is `copper_prefetch_unit_open`; larger local Vivado summaries are treated as existing evidence rather than portable rerun proof. The gem5 evidence is validated through retained summaries and provenance files where available [5]; the open-source full-core hardware harness uses PicoRV32 rather than a production ARM/OoO target [14].

7 Methodology

Table 3: Benchmarks

Class	Examples	Evidence file
Pointer-heavy	linked list, tree, hash, graph, Patricia	workload_build.csv; independent_sim_performance.csv
Controls	array, matrix, compute, random access	workload_build.csv; independent_sim_performance.csv
Stress	short chains, long chains, mixed, noisy, branchy	workload_build.csv; independent_sim_performance.csv

Table 4: Configurations And Baselines

Config	Role	Evidence file
no_prefetch	reference	cycle_performance.csv
next_line	spatial baseline	cycle_performance.csv
stride	regular-stream baseline	cycle_performance.csv
simple_pointer_chase	unsafe content-derived baseline	cycle_performance.csv
copper	committed-provenance policy	cycle_performance.csv

The normalized CSVs report per-workload rows rather than hiding negative results inside averages. Evidence levels are explicit: `model` rows come from the executable policy model, `cycle_model` rows come from a deterministic memory-system model with hit/miss latency, memory latency, outstanding prefetches, queue drops, lateness, and demand/prefetch traffic accounting. `core_integrated` rows add a deterministic core envelope with fetch/issue width, reorder-window pressure, load-queue pressure, branch penalties, and memory-system timing. `independent_sim` rows execute the source-built C workload driver for checksums and then use a separate trace/event cache simulator that does not import the cycle-model or core-integrated harness. `gem5_full_system` rows are imported from validated ARM-system summaries only when the compared group has a no-prefetch baseline, a COPPER-family row, matching checksums, zero return codes, and positive tick counts. `gem5_statistical_summary.csv` reports imported-summary confidence intervals where repeated samples exist and marks single-sample rows explicitly. This paper does not claim a clone-local rerun of every raw simulator run. None of these rows claim complete CPU integration.

8 Evaluation

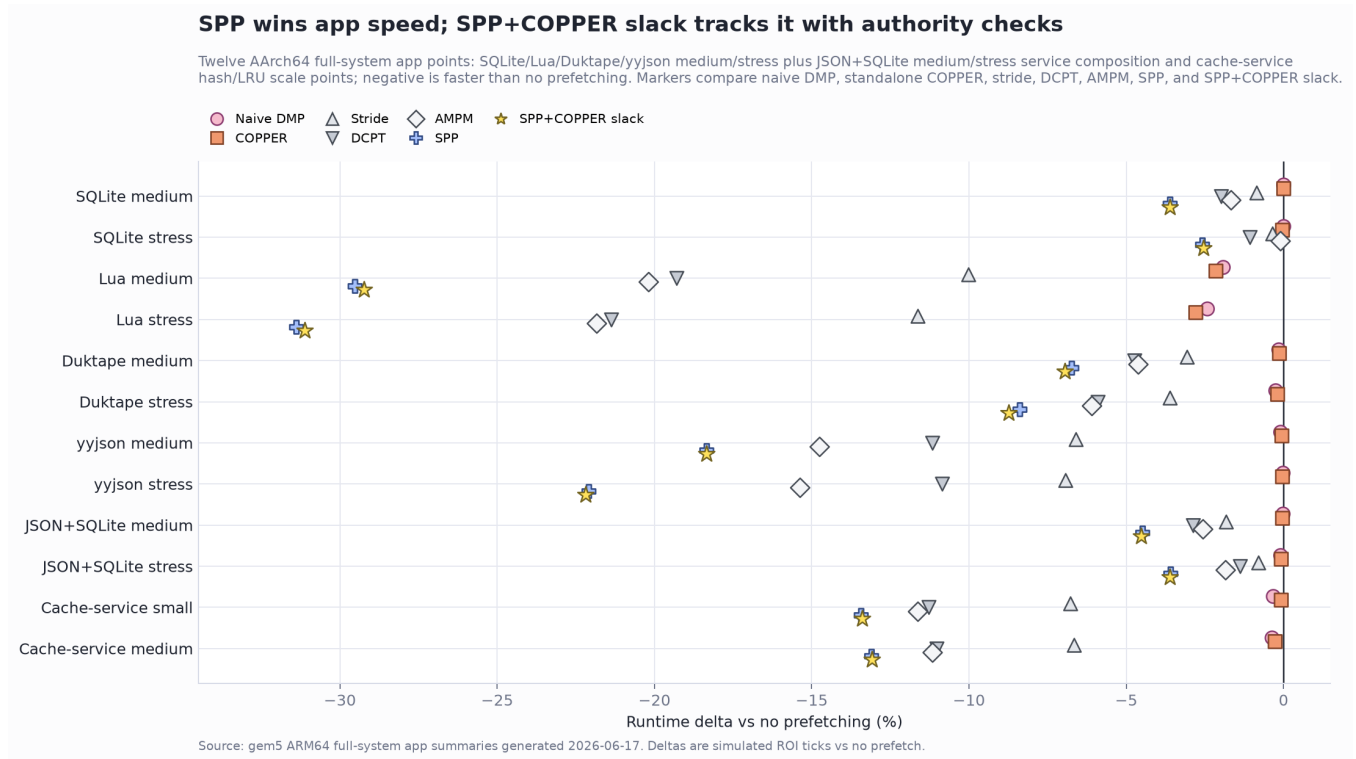


Figure 4: Application runtime deltas from the generated AArch64 application figure set. The plot keeps conventional SPP, SPP+COPPER slack, standalone COPPER, and unsafe data-derived baselines visible so authority and speed are discussed separately.

COPPER sharply reduces target-line witness misses

Reduction is computed against naive DMP on the same workload; higher is better. No translation faults were observed.

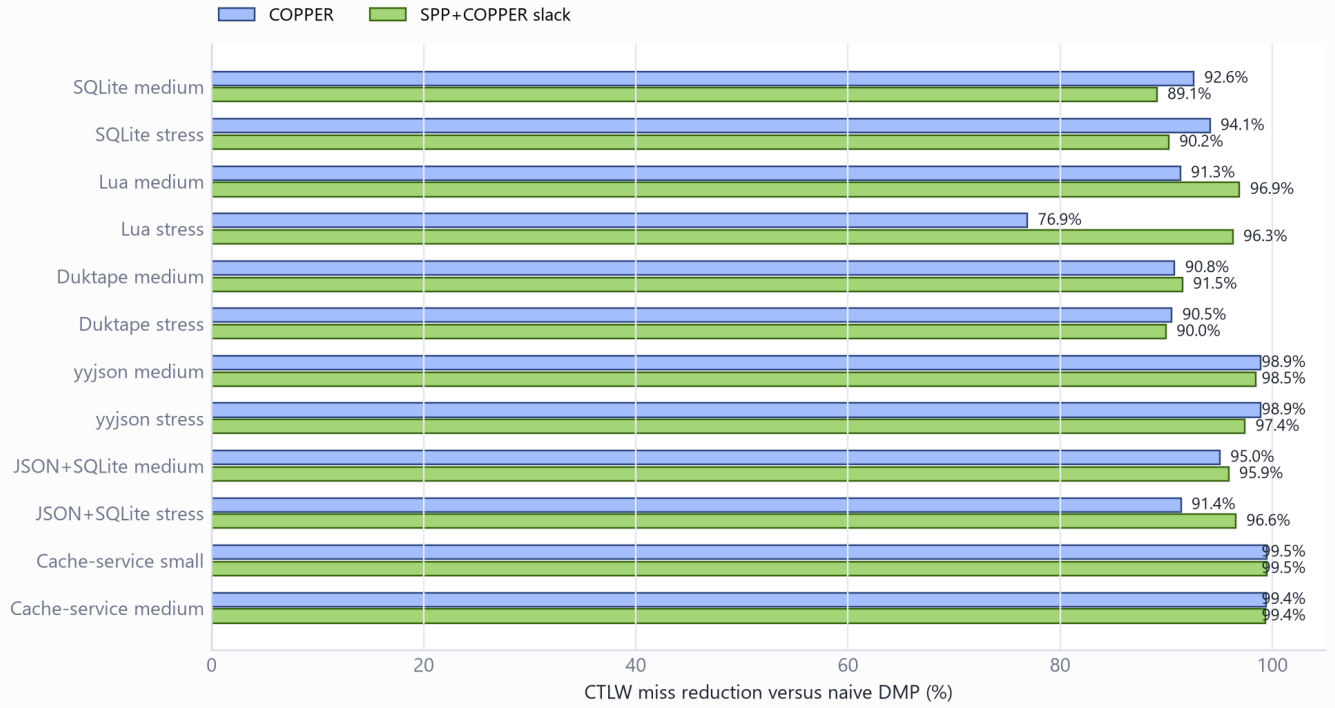


Figure 5: Target-line witness miss reduction against the unsafe data-derived baseline. This figure supports the authority-safety discussion rather than a universal performance claim.

Standalone COPPER keeps bus-byte deltas small on app workloads

Twelve full-system AArch64 app points. Bars show memory-bus byte delta for COPPER CLPD-64K+PEB versus no prefetching.

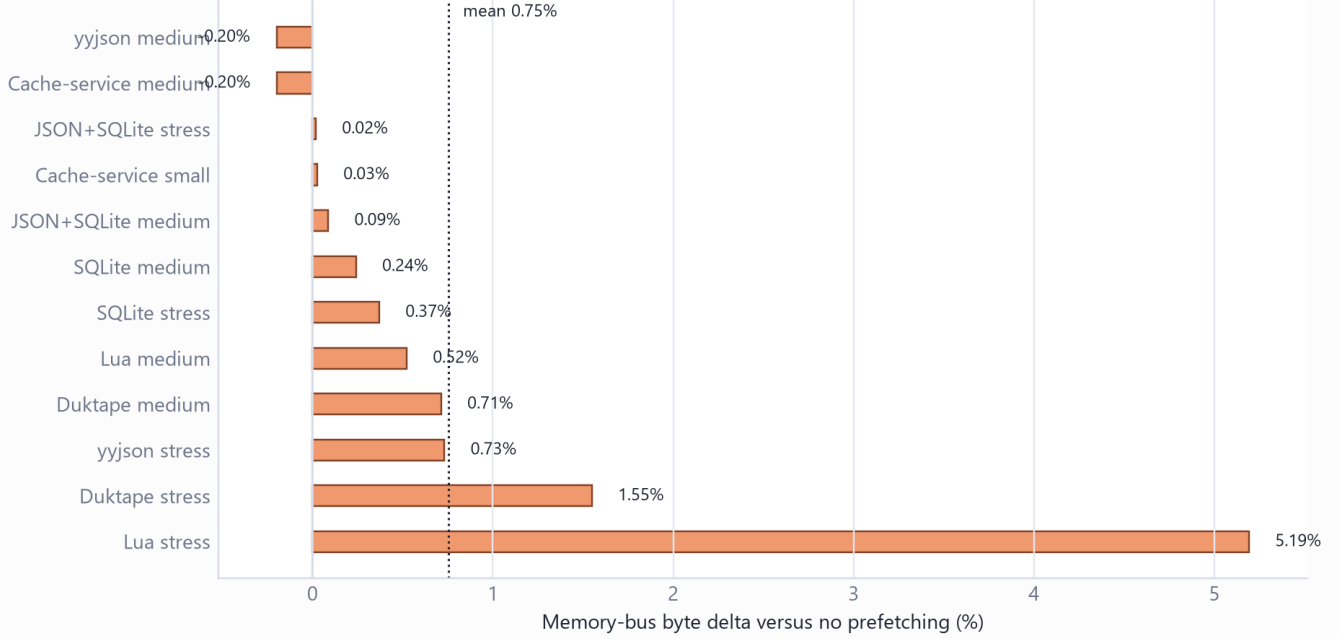


Figure 6: Memory-bus byte deltas for standalone COPPER on the generated application workload points. The figure is used only for evidence-level traffic discussion and is not a silicon-power measurement.

Table 5: Result Figure Provenance

Figure	Source image	Data and producer
4	copper_app_full_baseline_runtime.png	copper_prefetch_traffic_overhead_20260616.csv;
5	copper_app_ctlw_reduction.png	plot_copper_app_full_baseline_runtime.py
6	copper_app_bus_overhead.png	copper_prefetch_traffic_overhead_20260616.csv; plot_copper_app_overhead_figures.py
		copper_prefetch_traffic_overhead_20260616.csv; plot_copper_app_overhead_figures.py;
		COPPER_APP_FIGURE_INDEX_20260616.md

Current evidence supports selective, per-workload discussion. Several cycle-model, core-integrated, and independent-sim rows show COPPER behind the best baseline; those regressions remain in the CSVs and block a universal speedup claim.

9 Ablation And Sensitivity

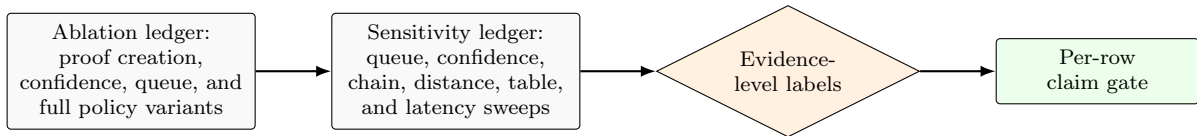


Figure 7: Ablation and sensitivity evidence flow. The CSV ledgers preserve scope labels so cycle-model counters are not presented as gem5 or hardware counters.

The cycle-model ablations isolate no provenance, speculative provenance, committed-only proof, confidence, queue filtering, and full COPPER. Queue size, confidence threshold, chain depth, distance, table size, and memory latency are varied in sensitivity rows. These are cycle-model counters and should not be described as gem5 counters.

10 Hardware Cost

Table 6: Hardware Cost

Evidence	Scope	File
CI RTL simulation	open-source unit test	rtl_simulation.csv
Yosys	generic unit resource context	synthesis.csv
nextpnr	mapped unit resource context when available	synthesis.csv
Overhead	matched unit rows from same flow	synthesis_overhead.csv
Near-core stub	matched generic near-core-stub rows when Yosys runs	fullcore_synthesis_overhead.csv
PicoRV32 core-wrapper	matched generic accepted core-wrapper rows when Yosys runs	fullcore_synthesis_overhead.csv
PicoRV32 tiny-SoC full-core	matched full-core rows when Yosys or mapped flows run	fullcore_synthesis_overhead.csv; mapped_ppa.csv
Mapped PPA	matched near-core-stub, PicoRV32 core-wrapper, PicoRV32 tiny-SoC full-core, or unit rows only when place-and-route succeeds	mapped_ppa.csv; mapped_ppa_overhead.csv
Production-core integration	not claimed	top_tier_gate_status.csv

The artifact supports unit-level hardware plausibility, matched near-core-stub resource-overhead rows, matched PicoRV32 accepted-core-wrapper rows, and matched PicoRV32 tiny-SoC full-core rows when the generated ledgers mark them PASS. The near-core stub is not a full CPU, the accepted wrapper is not full-core, and the PicoRV32 tiny-SoC full-core harness is not a production ARM/OoO integration. Generic Yosys rows do not provide mapped timing, Fmax, ASIC area, or measured power. Mapped timing may be discussed only when `mapped_ppa.csv` contains matched PASS rows from nextpnr, Vivado, or OpenROAD.

11 Energy Proxy

Table 7: Energy/Power Evidence

Evidence	Scope	File
Memory traffic proxy	assumption-based, not measured	energy_proxy.csv
ASIC Liberty tool-power index	Nangate45 standard-cell estimate when indexed PASS	asic_power.csv; power_report_index.csv
OpenROAD post-route power index	Nangate45 post-route tool estimate with indexed final DEF/SPEF/netlist when PASS	openroad_postroute_power.csv; power_report_index.csv
FPGA tool-power index	Vivado FPGA tool estimate when indexed PASS	power_report_index.csv; mapped_ppa.csv
Activity proxy	McPAT proxy when indexed PASS	power_report_index.csv; copper_mcpat_sensitivity_20260618.csv
Summary	proxy overhead statistics	energy_summary.csv

Energy rows use explicit assumptions recorded in `energy_proxy.csv`. When the power index marks OpenROAD post-route tool estimates as PASS, the row is a Nangate45 OpenROAD-flow-scripts post-route estimate with indexed final DEF/SPEF/netlist artifacts, not silicon measurement or foundry signoff. When ASIC Liberty tool estimates are PASS, the row is a Nangate45 standard-cell Liberty estimate from OpenSTA/OpenROAD, not silicon measurement

or post-route signoff with extracted parasitics. When FPGA tool estimates are PASS, the row is Vivado report power for the mapped FPGA target, not silicon or ASIC signoff. Activity-proxy PASS rows are fixed-architecture McPAT sensitivity runs driven by measured gem5 ROI counters. This supports evidence-level tool-power and proxy/model energy discussion, not foundry-signoff or silicon power-efficiency claims.

12 Limitations

Table 8: Limitations

Limitation	Consequence
Gem5 rows are validated imported ARM-system summaries	Gem5 validation applies to imported PASS rows
Independent simulator is trace/event level	It is not a replacement for a broad gem5 campaign
PicoRV32 tiny-SoC is the open-source full-core target	Production ARM/OoO claims remain blocked
Power is tool-estimate/model based	Signoff and silicon power-efficiency claims remain blocked
External gem5 and Vivado setup	Large external-tool reruns are not clone-local

13 Artifact Evidence Gates

Table 9: Artifact And Evidence Gates

Gate class	Status source	Interpretation
CI RTL evidence	rtl.simulation.csv	GitHub Actions unit simulation
Cycle-model evidence	cycle.performance.csv	deterministic memory-system timing model
Core-integrated evidence	core.integrated_performance.csv	deterministic core-envelope model
Independent simulator	independent_sim_performance.csv	source-backed trace/event simulator
Gem5 validated summaries	gem5.validation.csv; gem5.performance.csv; gem5.statistical_summary.csv	imported ARM-system rows and imported-summary statistics when PASS
Hardware cost	synthesis_overhead.csv; fullcore_synthesis_overhead.csv; mapped_ppa.csv	matched unit, near-core-stub, PicoRV32 core-wrapper, or PicoRV32 tiny-SoC full-core resources, plus mapped timing only when PASS
Energy proxy	energy_proxy.csv; openroad_postroute_power.csv; asic_power.csv; power_report_index.csv	assumption-based proxy plus OpenROAD post-route, ASIC Liberty, or FPGA tool estimates when indexed PASS
Package	artifact_manifest.csv	included and excluded files

14 Conclusion

COPPER is best framed as a committed-provenance authority mechanism for data-derived prefetch issue. The artifact supports that claim with a CI-proven open-source path, deterministic cycle-model and core-integrated evidence, source-backed independent-simulator rows, validated imported gem5 ARM-system rows with imported-summary statistics, matched unit-level, near-core-stub, PicoRV32 accepted-core-wrapper, and PicoRV32 tiny-SoC full-core FPGA PPA paths, FPGA tool-estimated power where indexed PASS, proxy energy rows, a claim ledger, audits, and a package manifest. The honest next step for broader production/silicon claims is a final raw-rerunnable gem5 or comparable external core validation matrix plus signoff-calibrated or silicon-measured power evidence without changing the evidence-bounded claims in this artifact.

References

- [1] Augury: Using data memory-dependent prefetchers to leak data at rest. <https://www.prefetchers.info/augury.pdf>.
- [2] Cheri: Capability hardware enhanced risc instructions. <https://www.cl.cam.ac.uk/research/security/ctsrtd/cheri/>.
- [3] Gofetch. <https://gofetch.fail/>.
- [4] Jean-Loup Baer and Tien-Fu Chen. An effective on-chip preloading scheme to reduce data access penalty. In *ACM/IEEE Supercomputing Conference*, 1991.
- [5] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, et al. The gem5 simulator. *ACM SIGARCH Computer Architecture News*, 2011.
- [6] Tien-Fu Chen and Jean-Loup Baer. Effective hardware-based data prefetching for high-performance processors. *IEEE Transactions on Computers*, 1995.
- [7] Intel. Data dependent prefetcher. <https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/technical-documentation/data-dependent-prefetcher.html>, 2022.
- [8] Norman P. Jouppi. Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers. In *International Symposium on Computer Architecture*, 1990.
- [9] Khaled N. Khasawneh, Esmail Mohammadian Koruyeh, Chengyu Song, Dmitry Evtushkin, Dmitry Ponomarev, and Nael Abu-Ghazaleh. Safespec: Banishing the spectre of a meltdown with leakage-free speculation. In *Design Automation Conference*, 2019.
- [10] Amir Roth, Andreas Moshovos, and Gurindar S. Sohi. Dependence based prefetching for linked data structures. In *International Conference on Architectural Support for Programming Languages and Operating Systems*, 1998.
- [11] Till Schlueter and Nils Ole Tippenhauer. Prefence: A fine-grained and scheduling-aware defense against prefetching-based attacks. In *IEEE European Symposium on Security and Privacy*, 2025.
- [12] Reshabh K. Sharma, Dan Grossman, and David Kohlbrenner. Splittingsecrets: A compiler-based defense for preventing data memory-dependent prefetcher side-channels. arXiv:2601.12270, 2026.
- [13] Santhosh Srinath, Onur Mutlu, Hyesoon Kim, and Yale N. Patt. Feedback directed prefetching: Improving the performance and bandwidth-efficiency of hardware prefetchers. In *International Symposium on High-Performance Computer Architecture*, 2007.
- [14] Clifford Wolf. Picorv32: A size-optimized risc-v cpu. <https://github.com/YosysHQ/picorv32>, 2019.